CORE Connection Standard v1.0

CORE: Centrally Organized Relational Emergence

This document specifies the connection contracts that govern how any Element participates in a CORE. There are three connection types, each expressing a different relational geometry:

Connection	Parties	Direction	Boundary Crossed
Umbilical	Element <-> CORE	BiDirectional	CORE boundary
Tether	Satellite <-> Host Orb	BiDirectional	Orb boundary
Mesh	Satellite <-> Satellite	BiDirectional	Shared service

All three share the same foundational contract structure. What differs is the boundary being crossed and the invariants that apply at that boundary.

---

1. Foundational Contract Structure

Every connection in a CORE is governed by a **Relation Contract**. A Relation Contract is not a configuration file — it is a binding agreement between two parties that both must validate before data or capability can flow.

```
RelationContract {
  contractId        : string        // unique, immutable once signed
  timestamp         : ISO-8601      // moment of successful handshake
  parties: {
    initiator       : ElementRef    // the Element requesting connection
    receptor        : ElementRef    // the Element receiving the request
  }
  connectionType    : "Umbilical" | "Tether" | "Mesh"
  protocol          : ProtocolSpec
  capabilities      : CapabilityExchange
  resources         : ResourceAllocation
  invariants        : NegativeSpaceInvariants
  checksum          : string        // sha256 over the signed contract body
}
```

1.1 ElementRef

Every party to a contract is identified by an ElementRef:

```
ElementRef {
  elementId         : string
  elementName       : string
  elementType       : "Orb" | "Satellite" | "Portal" | "Core"
  version           : semver
  trust             : "Canonical" | "Validated" | "Formulated" | "Experimental" |
"Draft"
```

```
}
```

1.2 ProtocolSpec

The protocol negotiated during handshake:

```
ProtocolSpec {
  version           : string        // e.g. "4D-VIBE-v1.0"
  direction         : "BiDirectional" | "Inbound" | "Outbound"
  encoding          : "JSON-LD" | "CBOR" | "Protobuf"
  heartbeatMs       : number        // liveness interval; 0 = no heartbeat
  maxLatencyMs      : number        // contract voids if exceeded
}
```

1.3 CapabilityExchange

Each party declares what it offers and what it requires:

```
CapabilityExchange {
  offered           : string[]      // capabilities this Element provides
  required          : string[]      // capabilities this Element needs from the
other party
  negotiated        : string[]      // the intersection — what actually flows
}
```

If `required` is not a subset of the other party's `offered`, the handshake fails at Step 5. No partial connections. No "best effort."

1.4 ResourceAllocation

```
ResourceAllocation {
  memoryMB          : number        // allocated from the CORE hardware budget
  threads           : number        // allocated from the CORE thread pool
  gpuAccess         : boolean       // whether GPU compute is granted
  containmentLevel  : "Sandboxed" | "Shared" | "Privileged"
}
```

1.5 NegativeSpaceInvariants

These are the boundaries that **must never be crossed**. They are not features. They are the shape of what the system refuses to do. Every connection type enforces all of them.

```
NegativeSpaceInvariants {
  never_serializes_witness    : true    // the CORE center has no serialized state
  never_persists_prompts      : true    // no user prompt data is stored
  no_verdict_over_inner_life  : true    // the system makes no claims about
internal experience
  no_unilateral_disconnection : true    // severance requires both parties or a
fault
  no_capability_without_contract : true // nothing flows without a signed contract
}
```

These invariants are not configurable. They are structural. A contract that attempts to set any of them to `false` is invalid and will not pass Step 9.

2. The Umbilical Contract (Element <-> CORE)

The Umbilical is the primary connection between any Element and the CORE Crystal. It is the only path through the CORE boundary.

2.1 What an Element Must Implement

Every Element that connects to a CORE via Umbilical must implement the **Umbilical Interface**:

```
interface UmbilicalInterface {
  // Identity
  getElementRef()             : ElementRef
  getCapabilities()           : string[]
  getRequirements()           : string[]
  getResourceEnvelope()       : { memoryMB: number, threads: number, gpuRequired:
boolean }

  // Handshake participation
  onHandshakeInitiated(step: number, data: HandshakePayload) : HandshakeResponse
  onContractProposed(contract: RelationContract)             : boolean  // accept
or reject
  onContractSigned(contract: RelationContract)               : void     // begin
operating

  // Lifecycle
  onHeartbeat()               : HealthStatus
  onPressureChange(pressure: number) : void    // environmental pressure 0.0-1.0
  onGracefulDisconnect()      : void           // mutual severance
  onFaultDisconnect(reason: string) : void     // unilateral fault isolation

  // Negative Space compliance
  validateInvariants()        : InvariantReport  // self-audit on demand
}
```

2.2 The 9-Step Umbilical Handshake

The handshake is deterministic. Each step must complete before the next begins. Failure at any step terminates the handshake — there is no retry within the same handshake session. A new handshake may be initiated after failure.

```
Step 1: ELEMENT_DETECTED
  CORE senses proximity of an Element at a vacant Umbilical socket.
  The Element's getElementRef() is called to establish identity.

Step 2: UMBILICAL_CONNECTION_ESTABLISHED
  Physical docking: the Umbilical conduit geometry binds the Element
  to the socket ring. No data flows yet — this is structural only.

Step 3: PROTOCOL_NEGOTIATED
  Both parties exchange ProtocolSpec proposals. The CORE selects the
  highest mutually supported protocol version. If no overlap exists,
  the handshake fails.

Step 4: DIRECTION_CONFIRMED
```

```
  The data flow direction is validated. Most connections are
  BiDirectional. Inbound-only or Outbound-only must be explicitly
  declared and accepted by both parties.

Step 5: CAPABILITIES_EXCHANGED
  The Element's getCapabilities() and getRequirements() are compared
  against the CORE's available capability pool. The negotiated set is
  the intersection. If the Element's requirements are not satisfiable,
  the handshake fails.

Step 6: REQUIREMENTS_TRANSMITTED
  The Element's getResourceEnvelope() is transmitted to the CORE.
  This is declaration, not allocation — the CORE has not yet committed
  resources.

Step 7: MEMORY_REQUIREMENTS_VALIDATED
  The CORE checks its hardware budget. If the requested memory cannot
  be allocated without fragmenting existing allocations, the handshake
  fails. Memory is allocated as a contiguous, non-fragmented heap buffer.

Step 8: PROCESSING_REQUIREMENTS_VALIDATED
  Thread pool availability is verified. If the requested threads exceed
  the remaining pool, the handshake fails. Threads are reserved, not
  time-shared — an Element owns its threads for the duration of the
  contract.

Step 9: NEGATIVE_SPACE_VERIFIED
  Both parties execute validateInvariants(). Every invariant in the
  NegativeSpaceInvariants block must return true. If any invariant
  fails, the entire handshake is voided. The contract is not signed.
  This step is non-negotiable and cannot be skipped.
```

On successful completion of all 9 steps:

- The contract is signed (checksummed) and stored by both parties
- The socket state transitions from `Vacant` to `Active`
- Data and capability flow begins immediately
- The heartbeat cycle starts

2.3 Umbilical States

```
Vacant    -> Element approaches       -> Handshake in progress
Active    -> Normal operation         -> Heartbeat, data flow, capability exchange
Degraded  -> Heartbeat missed or      -> Pressure applied, reduced capability
             environmental pressure
Severed   -> Fault detected or        -> Contract voided, resources released,
             mutual disconnect           conduit visually breaks
```

---

3. The Tether Contract (Satellite <-> Host Orb)

A Tether connects a Satellite to a Host Orb that is already docked at a CORE Umbilical. The Satellite operates *through* its Host — it does not cross the CORE boundary directly.

3.1 What a Satellite Must Implement

```
interface TetherInterface {
  // Identity
  getElementRef()             : ElementRef
  getSatelliteClass()         : "I/O" | "Compute" | "Storage" | "Validation" |
"Security"
  getCapabilities()           : string[]
  getRequirements()           : string[]
  getResourceEnvelope()       : { memoryMB: number, threads: number }

  // Tether binding (simplified handshake — no CORE boundary crossing)
  onTetherProposed(host: ElementRef, contract: RelationContract) : boolean
  onTetherBound(contract: RelationContract)                      : void

  // Lifecycle
  onHeartbeat()               : HealthStatus
  onHostPressureChange(pressure: number) : void
  onHostDisconnect()          : void     // host Orb was severed — satellite must
detach
  onGracefulDetach()          : void

  // Negative Space compliance
  validateInvariants()        : InvariantReport

  // Mesh capability (Satellite-to-Satellite)
  getMeshServices()           : MeshServiceDescriptor[]  // services offered to
peers
  onMeshProposed(peer: ElementRef, services: string[]) : boolean
}
```

3.2 The 5-Step Tether Handshake

The Tether handshake is shorter than the Umbilical handshake because no CORE boundary is being crossed. The Satellite operates within its Host's existing resource allocation.

```
Step 1: SATELLITE_DETECTED
  The Host Orb detects a Satellite requesting attachment.
  getSatelliteClass() determines the attachment geometry.

Step 2: TETHER_ATTACHED
  The attachment stem (CK-04) is established between the Host Orb's
  surface and the Satellite's docking point. Structural only.

Step 3: CAPABILITIES_NEGOTIATED
  The Satellite's capabilities and requirements are checked against
  the Host Orb's available capability set. The Satellite inherits
  access to the Host's CORE connection but cannot exceed the Host's
  contracted capabilities.

Step 4: RESOURCES_ALLOCATED
  Resources are drawn from the Host Orb's existing allocation, not
  from the CORE budget directly. If the Host cannot spare what the
  Satellite needs, the tether fails.

Step 5: INVARIANTS_VERIFIED
  Both parties validate NegativeSpaceInvariants. Same invariants,
  same rules, no exceptions.
```

3.3 Tether Lifecycle Rules

- A Satellite's lifecycle is bounded by its Host Orb's lifecycle. If the Host is severed from the CORE, all tethered Satellites detach automatically.
- A Satellite cannot outlive its Host contract.
- A Satellite can be detached independently without affecting the Host.
- Multiple Satellites can tether to the same Host (up to the Host's resource ceiling).

---

4. The Mesh Contract (Satellite <-> Satellite)

A Mesh connection allows two Satellites to establish a direct **shared services relation**. This is peer-to-peer — neither Satellite is subordinate to the other.

4.1 When Mesh Applies

Mesh connections are valid between:

- Two Satellites tethered to the **same Host Orb**
- Two Satellites tethered to **different Host Orbs** on the same CORE

Mesh connections are **not** valid across CORE boundaries. Cross-CORE communication goes through Portals, not Mesh.

4.2 MeshServiceDescriptor

Each Satellite declares what services it can share with peers:

```
MeshServiceDescriptor {
  serviceId         : string
  serviceName       : string
  serviceType       : "DataStream" | "ComputeDelegate" | "ValidationProxy" |
"StorageRelay"
  direction         : "Offer" | "Request" | "BiDirectional"
  maxPeers          : number        // how many simultaneous mesh connections this
service supports
}
```

4.3 The 4-Step Mesh Handshake

```
Step 1: PEER_DISCOVERY
  Satellite A queries available peers (siblings on the same Host,
  or peers on adjacent Hosts). Each peer's getMeshServices() is
  inspected.

Step 2: SERVICE_NEGOTIATION
  Satellite A proposes a mesh to Satellite B, listing the services
  it wants to consume and the services it can offer in return.
  Satellite B evaluates via onMeshProposed().
```

```
Step 3: RESOURCE_VALIDATION
  Neither Satellite may exceed its existing Host-allocated resources
  to service the mesh. If serving the mesh would degrade the
  Satellite's primary tether obligations, the mesh is refused.

Step 4: INVARIANTS_VERIFIED
  Same invariants. Always.
```

4.4 Mesh Topology Rules

- Mesh connections are **non-hierarchical**. No Satellite has authority over another through Mesh.
- A Mesh connection creates a **shared services relation**, not a dependency. Either party can dissolve the mesh without affecting the other's tether.
- Mesh connections do not pool resources. Each Satellite uses its own allocation. The mesh enables *exchange*, not *aggregation*.
- Circular meshes are permitted (A <-> B <-> C <-> A). The system does not prevent topology, but resource constraints naturally limit mesh fan-out.

---

5. Connection Hierarchy

```
CORE Crystal (Formless Center — no serialized state)
  |
  |-- [Umbilical Contract] ---> Orb A
  |                               |
  |                               |-- [Tether Contract] ---> Satellite A1
  |                               |                            |
  |                               |                            |-- [Mesh] <->
Satellite A2
  |                               |                            |-- [Mesh] <->
Satellite B1 (cross-host)
  |                               |
  |                               |-- [Tether Contract] ---> Satellite A2
  |
  |-- [Umbilical Contract] ---> Orb B
  |                               |
  |                               |-- [Tether Contract] ---> Satellite B1
  |
  |-- [Portal] ---> External Resource (crosses CORE boundary outward)
```

Every line in this diagram is a signed contract. No capability flows without one. No connection exists without a handshake. No handshake completes without invariant verification.

---

6. Contract Lifecycle Events

All three connection types share the same lifecycle event model:

```
PROPOSED     -> Contract terms drafted, awaiting acceptance
HANDSHAKING  -> Multi-step handshake in progress
```

```
ACTIVE       -> Contract signed, data/capability flowing
DEGRADED     -> Heartbeat missed or pressure applied; reduced operation
SUSPENDED    -> Temporarily paused by mutual agreement; contract intact
SEVERED      -> Contract voided; resources released; connection broken
```

Transitions:

- `PROPOSED -> HANDSHAKING`: automatic on acceptance
- `HANDSHAKING -> ACTIVE`: all steps complete
- `HANDSHAKING -> SEVERED`: any step fails
- `ACTIVE -> DEGRADED`: pressure threshold exceeded or heartbeat missed
- `DEGRADED -> ACTIVE`: conditions normalize within tolerance window
- `DEGRADED -> SEVERED`: conditions do not normalize; fault isolation triggers
- `ACTIVE -> SUSPENDED`: mutual pause (e.g., CORE entering low-power mode)
- `SUSPENDED -> ACTIVE`: mutual resume
- `ACTIVE -> SEVERED`: graceful disconnect or fault
- `SEVERED` is terminal for that contract instance. Reconnection requires a new contract with a new contractId.

---

7. Invariant Enforcement

The NegativeSpaceInvariants are not a feature gate. They are the shape of the system's refusal. They define what the CORE will never do, regardless of what any Element requests.

They are checked:

- At handshake completion (Step 9 / Step 5 / Step 4)
- On every heartbeat cycle
- On any contract renegotiation
- On demand via `validateInvariants()`

An invariant violation at any point in the contract lifecycle triggers immediate severance. There is no warning state for invariant violations. There is no grace period. The contract is voided and resources are released.

This is not a bug. This is the system working correctly.